Assignment 1:

SET A

1. Write a Python program to read an entire text file.

In [5]:
```
file = open("example.txt", "r")

# Read the entire content
content = file.read()

file.close()

print(content)
```

```
How are you?
```

2. Write a Python program to compute the number of characters, words and lines in a file.

In [6]:
```
file = open("example.txt", "r")

num_chars = 0
num_words = 0
num_lines = 0

# Read lines one by one
for line in file:
    num_lines += 1
    num_chars += len(line)
    num_words += len(line.split())

file.close()

print("Number of lines:", num_lines)
print("Number of words:", num_words)
print("Number of characters:", num_chars)
```

```
Number of lines: 1
Number of words: 3
Number of characters: 12
```

3. Write a Python script to print the current date in following format “Sun May 29 02:26:23 IST 2017”

In [7]:
```python
import time

# Get current time in desired format
current_time = time.strftime("%a %b %d %H:%M:%S IST %Y")

print(current_time)
```

```
Sun Dec 07 16:26:41 IST 2025
```

SET B

1. Write a Python program to append text to a file and display the text.

In [8]:
```python
# Append text to the file
file = open("example.txt", "a")
file.write("This is new content added.\n")
file.close()

# Read and display the updated file content
file = open("example.txt", "r")
content = file.read()
file.close()

print("Updated File Content:\n")
print(content)
```

```
Updated File Content:

How are you?This is new content added.
```

2. Write a Python program to print each line of a file in reverse order.

In [9]:
```python
file = open("example.txt", "r")
```

```
lines = file.readlines()

file.close()

# Print each line reversed
for line in lines:
    print(line[::-1])
```

```
.dedda tnetnoc wen si sihT?uoy era woH
```

3. Write a Python program to print date, time for today and now.

In [10]:
```
import datetime

# Get today's date
today_date = datetime.date.today()

# Get current date and time
now_datetime = datetime.datetime.now()

print("Today's Date:", today_date)
print("Current Date & Time:", now_datetime)
print("Current Time:", now_datetime.time())
```

```
Today's Date: 2025-12-07
Current Date & Time: 2025-12-07 16:47:42.433599
Current Time: 16:47:42.433599
```

PROGRAMS FOR PRACTICE:

1. Write a Python program to read an entire text file.

In [11]:
```
# Open the file in read mode
file = open("example.txt", "r")

content = file.read()

file.close()

print(content)
```

```
How are you?This is new content added.
```

2. Write a Python program to read first n lines of a file.

In [16]:
```python
n = 3
file = open("example.txt", "r")

for i in range(n):
    line = file.readline()
    if not line:
        break
    print(line, end="")

file.close()
```

```
How are you?This is new content added.
```

3. Write a Python program to append text to a file and display the text.

In [17]:
```python
file = open("example.txt", "a")
file.write("This is appended text.\n")
file.close()

file = open("example.txt", "r")
content = file.read()
file.close()

print("Updated File Content:\n")
print(content)
```

```
Updated File Content:

How are you?This is new content added.
This is appended text.
```

4. Write a Python program to read last n lines of a file.

In [18]:
```python
n = 3
```

```python
file = open("example.txt", "r")

lines = file.readlines()

file.close()

# Print last n lines
for line in lines[-n:]:
    print(line, end="")
```

```
How are you?This is new content added.
This is appended text.
```

5. Write a Python program to read a file line by line and store it into a list.

In [19]:
```python
file = open("example.txt", "r")

lines_list = []

for line in file:
    lines_list.append(line.strip())

file.close()

print(lines_list)
```

```
['How are you?This is new content added.', 'This is appended text.']
```

6. Write a Python program to read a file line by line store it into a variable.

In [20]:
```python
file = open("example.txt", "r")

content = ""

for line in file:
    content += line   # add each line to the variable

file.close()

print(content)
```

```
How are you?This is new content added.
This is appended text.
```

7. Write a Python program to read a file line by line store it into an array.

In [21]:
```python
file = open("example.txt", "r")

arr = []

for line in file:
    arr.append(line.strip())   # strip() removes \n

file.close()

print(arr)
```

```
['How are you?This is new content added.', 'This is appended text.']
```

1. Write a python program to find the longest words.

In [22]:
```python
file = open("example.txt", "r")

words = file.read().split()

file.close()

# Find the length of the longest word
max_length = len(max(words, key=len))

# Find all words with that length
longest_words = [word for word in words if len(word) == max_length]

print("Longest word(s):", longest_words)
print("Length:", max_length)
```

```
Longest word(s): ['you?This', 'appended']
Length: 8
```

2. Write a Python program to count the number of lines in a text file.

In [23]:
```python
file = open("example.txt", "r")

count = 0

for line in file:
    count += 1

file.close()

print("Number of lines:", count)
```

```
Number of lines: 2
```

3. Write a Python program to count the frequency of words in a file.

In [24]:
```python
file = open("example.txt", "r")

text = file.read()
file.close()

words = text.split()          # split text into words
freq = {}                     # dictionary to store word frequency

for word in words:
    word = word.lower()
    freq[word] = freq.get(word, 0) + 1

print("Word Frequency:\n")
for word, count in freq.items():
    print(word, ":", count)
```

```
Word Frequency:

how : 1
are : 1
you?this : 1
is : 2
new : 1
content : 1
added. : 1
this : 1
appended : 1
text. : 1
```

4. Write a Python program to get the file size of a plain file.

In [25]:
```python
import os

file_path = "example.txt"

size = os.path.getsize(file_path)

print("File size:", size, "bytes")
```

```
File size: 64 bytes
```

5. Write a Python program to write a list to a file.

In [26]:
```python
my_list = ["Apple", "Banana", "Cherry", "Date"]

file = open("example.txt", "w")

for item in my_list:
    file.write(item + "\n")

file.close()

print("List has been written to the file.")
```

```
List has been written to the file.
```

13. Write a Python program to copy the contents of a file to another file .

In [28]:
```
source_file = open("source.txt", "r")

destination_file = open("destination.txt", "w")

for line in source_file:
    destination_file.write(line)

source_file.close()
destination_file.close()

print("File copied successfully!")
```

```
File copied successfully!
```

14. Write a Python program to combine each line from first file with the corresponding line in second file.

In [30]:
```
file1 = open("file1.txt", "r")
file2 = open("file2.txt", "r")

lines1 = file1.readlines()
lines2 = file2.readlines()

file1.close()
file2.close()

# Combine corresponding lines
combined_lines = []
for line1, line2 in zip(lines1, lines2):
    combined_lines.append(line1.strip() + " " + line2.strip())

for line in combined_lines:
    print(line)
```

```
Hello,Good Evening Hello,Good Evening Everything
```

13. Write a Python program to remove newline characters from a file.

In [2]:
```python
file = open("input.txt", "r")
content = file.read()
file.close()

# Remove newline characters
content = content.replace("\n", "")

file = open("input.txt", "w")
file.write(content)
file.close()

print("Newline characters removed successfully.")
```

```
Newline characters removed successfully.
```

14. Write a Python program that takes a text file as input and returns the number of words of a given text file.

In [3]:
```python
file = open("input.txt", "r")
content = file.read()
file.close()

# Split the content into words
words = content.split()

# Count the number of words
word_count = len(words)

print("Number of words in the file:", word_count)
```

```
Number of words in the file: 2
```

15. Write a Python program to extract characters from various text files and puts them into a list.

In [5]:
```python
files = ["file1.txt", "file2.txt", "file3.txt"]

characters = []

# Read each file
for filename in files:
    file = open(filename, "r")
```

```
    content = file.read()
    file.close()

    # Add each character to the list
    for ch in content:
        characters.append(ch)

print("Characters extracted into list:")
print(characters)
```

```
Characters extracted into list:
['H', 'e', 'l', 'l', 'o', ',', 'G', 'o', 'o', 'd', ' ', 'E', 'v', 'e', 'n', 'i', 'n', 'g', 'H', 'e', 'l', 'l', 'o', ',', 'G',
'o', 'o', 'd', ' ', 'E', 'v', 'e', 'n', 'i', 'n', 'g', ' ', 'E', 'v', 'e', 'r', 'y', 't', 'h', 'i', 'n', 'g', 'W', 'r', 'i',
't', 'e', ' ', 'a', ' ', 'P', 'y', 't', 'h', 'o', 'n', ' ', 'p', 'r', 'o', 'g', 'r', 'a', 'm', ' ', 't', 'o', ' ', 'e', 'x',
't', 'r', 'a', 'c', 't', ' ', 'c', 'h', 'a', 'r', 'a', 'c', 't', 'e', 'r', 's', ' ', 'f', 'r', 'o', 'm', ' ', 'v', 'a', 'r',
'i', 'o', 'u', 's', ' ', 't', 'e', 'x', 't', ' ', 'f', 'i', 'l', 'e', 's', ' ', 'a', 'n', 'd', ' ', 'p', 'u', 't', 's', ' ',
't', 'h', 'e', 'm', ' ', 'i', 'n', 't', 'o', ' ', 'a', ' ', 'l', 'i', 's', 't', '.']
```

16. Write a python program to get Current Time

In [6]:
```
import datetime

current_time = datetime.datetime.now().time()
print("Current Time:", current_time)
```

```
Current Time: 11:51:59.896674
```

19. Get Current Date and Time using Python

In [7]:
```
import datetime

now = datetime.datetime.now()
print("Current Date and Time:", now)
```

```
Current Date and Time: 2025-12-20 11:53:23.351984
```

20. Write a python | Find yesterday's, today's and tomorrow's date

In [8]:
```
import datetime

today = datetime.date.today()
```

```
yesterday = today - datetime.timedelta(days=1)
tomorrow = today + datetime.timedelta(days=1)

print("Yesterday's Date:", yesterday)
print("Today's Date:", today)
print("Tomorrow's Date:", tomorrow)
```

```
Yesterday's Date: 2025-12-19
Today's Date: 2025-12-20
Tomorrow's Date: 2025-12-21
```

21. Write a python program to convert time from 12 hour to 24 hour format

In [10]:
```
import datetime

# Input time in 12-hour format
time_12 = input("Enter time (HH:MM:SS AM/PM): ")

# Convert to 24-hour format
time_24 = datetime.datetime.strptime(time_12, "%I:%M:%S %p")

print("24-hour format:", time_24.strftime("%H:%M:%S"))
```

```
24-hour format: 19:45:30
```

22. Write a python program to find difference between current time and given time

In [11]:
```
import datetime

# Get current date and time
current_time = datetime.datetime.now()

given_time = input("Enter time (HH:MM:SS): ")

# Convert given time to datetime (using today's date)
given_time = datetime.datetime.strptime(given_time, "%H:%M:%S")
given_time = given_time.replace(
    year=current_time.year,
    month=current_time.month,
    day=current_time.day
```

```
)

time_difference = current_time - given_time

print("Current Time:", current_time.strftime("%H:%M:%S"))
print("Given Time:", given_time.strftime("%H:%M:%S"))
print("Time Difference:", time_difference)
```

```
Current Time: 11:57:44
Given Time: 10:30:00
Time Difference: 1:27:44.034699
```

19. Convert date string to timestamp in Python

In [1]:
```
import datetime
import time

# Input date string
date_string = "2024-12-20 10:30:00"

# Convert string to datetime object
dt = datetime.datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")

# Convert datetime to timestamp
timestamp = dt.timestamp()

print("Date String:", date_string)
print("Timestamp:", timestamp)
```

```
Date String: 2024-12-20 10:30:00
Timestamp: 1734670800.0
```

25. Find number of times every day occurs in a Year

In [2]:
```
import datetime

year = int(input("Enter year: "))

# Dictionary to store count of days
days_count = {
    "Monday": 0,
```

```python
    "Tuesday": 0,
    "Wednesday": 0,
    "Thursday": 0,
    "Friday": 0,
    "Saturday": 0,
    "Sunday": 0
}

# Start and end date of the year
start_date = datetime.date(year, 1, 1)
end_date = datetime.date(year, 12, 31)

current_date = start_date

# Loop through each day of the year
while current_date <= end_date:
    day_name = current_date.strftime("%A")
    days_count[day_name] += 1
    current_date += datetime.timedelta(days=1)

print("Number of times each day occurs in year", year)
for day, count in days_count.items():
    print(day, ":", count)
```

```
Number of times each day occurs in year 2024
Monday : 53
Tuesday : 53
Wednesday : 52
Thursday : 52
Friday : 52
Saturday : 52
Sunday : 52
```

Assignment 2:

SET A

1. Write a Python program to demonstrate the zero division error and overflow error.

In [4]:
```python
# Demonstrating ZeroDivisionError
try:
    a = 10
    b = 0
    result = a / b
except ZeroDivisionError:
    print("ZeroDivisionError occurred: division by zero")

# Demonstrating OverflowError
try:
    import math
    result = math.exp(1000)   # Very large number
except OverflowError:
    print("OverflowError occurred: number too large")
```

```
ZeroDivisionError occurred: division by zero
OverflowError occurred: number too large
```

2. Write a Python program to find sequences of lowercase letters joined with a underscore

In [5]:
```python
import re

text = "valid_sequence another_valid one_more invalid_Sequence test_123"

# Find lowercase sequences joined with underscore
result = re.findall(r'\b[a-z]+_[a-z]+\b', text)

print("Matching sequences:")
print(result)
```

```
Matching sequences:
['valid_sequence', 'another_valid', 'one_more']
```

3. Write a python program to Check if String Contain Only Defined Characters using Regex

In [6]:
```python
import re

text = input("Enter a string: ")

# Define allowed characters (a-z, A-Z, 0-9)
```

```python
pattern = r'^[a-zA-Z0-9]+$'

# Check using regex
if re.match(pattern, text):
    print("The string contains only defined characters.")
else:
    print("The string contains invalid characters.")
```

```
The string contains only defined characters.
```

SET B

1. Write a Python program to match a string that contains only upper and lowercase letters, numbers, and underscores.Write a Python program to raised the attribute error, if attribute class object has no attribute with the name attribute.

In [7]:
```python
import re

# Input string
text = input("Enter a string: ")

# Regex pattern: only letters, numbers, and underscore
pattern = r'^[a-zA-Z0-9_]+$'

if re.match(pattern, text):
    print("Valid string: contains only letters, numbers, and underscores.")
else:
    print("Invalid string: contains other characters.")
```

```
Valid string: contains only letters, numbers, and underscores.
```

2. Write a python Program to Remove duplicate words from Sentence

In [1]:
```python
sentence = input("Enter a sentence: ")

# Split the sentence into words
words = sentence.split()

unique_words = []

# Iterate through words and add only unique ones
for word in words:
```

```
    if word not in unique_words:
        unique_words.append(word)

# Join the unique words back into a sentence
new_sentence = ' '.join(unique_words)

print("Sentence after removing duplicates:")
print(new_sentence)
```

```
Sentence after removing duplicates:
This is a test sentence
```

3. Write a python to| Remove all characters except letters and numbers

In [2]:
```
import re

text = input("Enter a string: ")

# Remove all characters except letters and numbers
cleaned_text = re.sub(r'[^a-zA-Z0-9]', '', text)

print("String after removing special characters:")
print(cleaned_text)
```

```
String after removing special characters:
HelloWelcometoPython310
```

PROGRAMS FOR PRACTICE:

1. Write a python program to Count Uppercase, Lowercase, special character and numeric values using Regex

In [3]:
```
import re

text = input("Enter a string: ")

# Count uppercase letters
uppercase_count = len(re.findall(r'[A-Z]', text))

# Count lowercase letters
lowercase_count = len(re.findall(r'[a-z]', text))
```

```python
# Count numeric digits
digit_count = len(re.findall(r'[0-9]', text))

# Count special characters (anything not letter or number)
special_count = len(re.findall(r'[^a-zA-Z0-9]', text))

print("Uppercase letters:", uppercase_count)
print("Lowercase letters:", lowercase_count)
print("Numeric digits:", digit_count)
print("Special characters:", special_count)
```

```
Uppercase letters: 2
Lowercase letters: 8
Numeric digits: 3
Special characters: 3
```

2. Write a python program to find the most occurring number in a string using Regex

In [4]:

```python
import re
from collections import Counter

text = input("Enter a string: ")

# Extract all numbers from the string
numbers = re.findall(r'\d+', text)  # \d+ matches sequences of digits

if numbers:
    # Count the frequency of each number
    count = Counter(numbers)

    # Find the most common number(s)
    most_common_number, frequency = count.most_common(1)[0]

    print(f"The most occurring number is: {most_common_number} (occurs {frequency} times)")
else:
    print("No numbers found in the string.")
```

```
The most occurring number is: 2 (occurs 2 times)
```

3. Write a python Regex to extract maximum numeric value from a string

In [5]:
```python
import re

text = input("Enter a string: ")

# Extract all numbers from the string
numbers = re.findall(r'\d+', text)  # \d+ matches sequences of digits

if numbers:
    # Convert all extracted numbers to integers
    numbers = [int(num) for num in numbers]

    max_number = max(numbers)

    print(f"The maximum numeric value in the string is: {max_number}")
else:
    print("No numbers found in the string.")
```

```
The maximum numeric value in the string is: 123
```

4. Write a python program to put spaces between words starting with capital letters using Regex

In [6]:
```python
import re

text = input("Enter a string: ")

# Insert space before each capital letter (except the first character)
new_text = re.sub(r'(?<!^)(?=[A-Z])', ' ', text)

print("String after adding spaces:")
print(new_text)
```

```
String after adding spaces:
Hello World This Is Python
```

4. Write a python to Check whether a string starts and ends with the same character or not

In [7]:
```python
text = input("Enter a string: ")

# Check if string is not empty
if len(text) > 0 and text[0] == text[-1]:
```

```
    print("The string starts and ends with the same character.")
else:
    print("The string does not start and end with the same character.")
```

```
The string starts and ends with the same character.
```

6. Write a python regex to find sequences of one upper case letter followed by lower case letters

In [8]:
```
import re

text = input("Enter a string: ")

# Regex pattern: one uppercase letter followed by one or more lowercase letters
pattern = r'[A-Z][a-z]+'

matches = re.findall(pattern, text)

print("Sequences found:")
print(matches)
```

```
Sequences found:
['Hello', 'World', 'This', 'Is', 'Python']
```

6. Write a python Regex program to accept string ending with alphanumeric character

In [9]:
```
import re

text = input("Enter a string: ")

# Regex pattern: ends with an alphanumeric character
pattern = r'.*[a-zA-Z0-9]$'

# Check if the string matches the pattern
if re.match(pattern, text):
    print("The string ends with an alphanumeric character.")
else:
    print("The string does NOT end with an alphanumeric character.")
```

```
The string ends with an alphanumeric character.
```

7. Write a python Regex program to accept string starting with vowel

In [10]:
```python
import re

text = input("Enter a string: ")

# Regex pattern: starts with a vowel (case-insensitive)
pattern = r'^[aeiouAEIOU]'

# Check if the string matches the pattern
if re.match(pattern, text):
    print("The string starts with a vowel.")
else:
    print("The string does NOT start with a vowel.")
```

```
The string starts with a vowel.
```

8. Write a python Program to check if a string starts with a substring using regex

In [11]:
```python
import re

text = input("Enter a string: ")

# Substring to check
substring = input("Enter the substring to check: ")

# Regex pattern: starts with the given substring
pattern = r'^' + re.escape(substring)  # re.escape handles special characters in substring

# Check if the string matches the pattern
if re.match(pattern, text):
    print(f"The string starts with '{substring}'.")
else:
    print(f"The string does NOT start with '{substring}'.")
```

```
The string starts with 'HelloWorld'.
```

9. Write a python Program to Check if an URL is valid or not using Regular Expression

In [12]:
```python
import re

url = input("Enter a URL: ")
```

```python
# Regex pattern for validating a URL
pattern = re.compile(
    r'^(https?|ftp)://'          # http, https, or ftp
    r'([A-Za-z0-9-]+\.)+'        # domain name
    r'[A-Za-z]{2,}'              # top-level domain
    r'(:\d+)?'                   # optional port
    r'(/.*)?$'                   # optional path
)

# Check if the URL matches the pattern
if re.match(pattern, url):
    print("Valid URL")
else:
    print("Invalid URL")
```

```
Valid URL
```

10. Write a python Program to Parsing and Processing URL using Python – Regex

In [13]:

```python
import re

url = input("Enter a URL: ")

# Regex pattern to parse URL
pattern = re.compile(
    r'^(?P<protocol>https?|ftp)://'   # protocol
    r'(?P<domain>[A-Za-z0-9.-]+)'     # domain
    r'(?::(?P<port>\d+))?'            # optional port
    r'(?P<path>/.*)?$'                # optional path
)

match = re.match(pattern, url)

if match:
    print("URL Parsing Successful")
    print("Protocol :", match.group('protocol'))
    print("Domain   :", match.group('domain'))
    print("Port     :", match.group('port'))
    print("Path     :", match.group('path'))
```

```
else:
    print("Invalid URL format")
```

```
URL Parsing Successful
Protocol : https
Domain   : www.example.com
Port     : 8080
Path     : /path/page.html
```

11. Write a python Program to validate an IP address using ReGex

In [14]:
```
import re

ip = input("Enter an IP address: ")

# Regex pattern for validating IPv4 address
pattern = re.compile(
    r'^('
    r'(25[0-5]|2[0-4][0-9]|1[0-9]{2}|[1-9]?[0-9])\.'
    r'){3}'
    r'(25[0-5]|2[0-4][0-9]|1[0-9]{2}|[1-9]?[0-9])$'
)

# Check if IP matches the pattern
if re.match(pattern, ip):
    print("Valid IP address")
else:
    print("Invalid IP address")
```

```
Valid IP address
```

12. Write a python Program to Check if email address valid or not

In [15]:
```
import re

email = input("Enter an email address: ")

# Regex pattern for validating email
pattern = re.compile(
    r'^[a-zA-Z0-9._%+-]+'   # username
    r'@'                    # @ symbol
```

```
    r'[a-zA-Z0-9.-]+'       # domain name
    r'\.[a-zA-Z]{2,}$'      # top-level domain
)

# Check if email matches the pattern
if re.match(pattern, email):
    print("Valid Email Address")
else:
    print("Invalid Email Address")
```

```
Valid Email Address
```

13. Write a python program to find files having a particular extension using RegEx

In [17]:
```
import re
import os

# Input directory path and file extension
directory = input("Enter directory path: ")
extension = input("Enter file extension (e.g., .txt): ")

# Create regex pattern for the given extension
pattern = re.compile(r'.*' + re.escape(extension) + r'$')

print(f"\nFiles with '{extension}' extension:")

# List and filter files using regex
for filename in os.listdir(directory):
    if re.match(pattern, filename):
        print(filename)
```

```
Files with '.txt' extension:
example.txt.txt
file 3.txt.txt
file1.txt.txt
file2.txt.txt
input.txt.txt
source.txt.txt
```

14. Write a python program to extract IP address from file

In [21]:
```python
import re

# Input file name
filename = input("Enter file name: ")

# Regex pattern for IPv4 address
ip_pattern = re.compile(
    r'\b(?:'
    r'(?:25[0-5]|2[0-4][0-9]|1[0-9]{2}|[1-9]?[0-9])\.'
    r'){3}'
    r'(?:25[0-5]|2[0-4][0-9]|1[0-9]{2}|[1-9]?[0-9])\b'
)

try:
    file = open(filename, 'r')
    content = file.read()
    file.close()

    # Extract IP addresses
    ip_addresses = ip_pattern.findall(content)

    if ip_addresses:
        print("IP addresses found:")
        for ip in ip_addresses:
            print(ip)
    else:
        print("No IP addresses found.")

except FileNotFoundError:
    print("File not found. Please check the file name.")
```

```
No IP addresses found.
```

16. Write a python program to check the validity of a Password Students can practice Common Examples of Exception as:
17. Division by Zero
18. Accessing a file which does not exist.
19. Addition of two incompatible types
20. Trying to access a nonexistent index of a sequence

21. Removing the table from the disconnected database server.
22. ATM withdrawal of more than the available amount

In [22]:
```
import re

password = input("Enter password: ")

pattern = re.compile(
    r'^(?=.*[A-Z])'      # at least one uppercase letter
    r'(?=.*[a-z])'      # at least one lowercase letter
    r'(?=.*\d)'         # at least one digit
    r'(?=.*[@$!%*?&])'  # at least one special character
    r'[A-Za-z\d@$!%*?&]{8,}$'  # minimum 8 characters
)

if re.match(pattern, password):
    print("Valid Password")
else:
    print("Invalid Password")
```

```
Valid Password
```

1. Division by Zero

In [23]:
```
try:
    a = 10 / 0
except ZeroDivisionError:
    print("Error: Division by zero is not allowed")
```

```
Error: Division by zero is not allowed
```

2. Accessing a File Which Does Not Exist

In [25]:
```
try:
    file = open("nofile.txt", "r")
    file.read()
    file.close()
except FileNotFoundError:
    print("Error: File does not exist")
```

3. Addition of Two Incompatible Types

In [26]:
```
try:
    result = 10 + "20"
except TypeError:
    print("Error: Cannot add integer and string")
```

```
Error: Cannot add integer and string
```

4. Accessing a Non-existent Index of a Sequence

In [27]:
```
try:
    list1 = [1, 2, 3]
    print(list1[5])
except IndexError:
    print("Error: Index out of range")
```

```
Error: Index out of range
```

5. Removing a Table from a Disconnected Database Server

In [28]:
```
try:
    raise ConnectionError("Database server not connected")
except ConnectionError as e:
    print("Error:", e)
```

```
Error: Database server not connected
```

6. ATM Withdrawal of More Than Available Amount

In [29]:
```
try:
    balance = 5000
    withdraw = int(input("Enter withdrawal amount: "))

    if withdraw > balance:
        raise ValueError("Insufficient balance")

    balance -= withdraw
    print("Withdrawal successful")
    print("Remaining balance:", balance)
```

```
except ValueError as e:
    print("Error:", e)
```

```
Withdrawal successful
Remaining balance: 4000
```

Assignment 3:

1) Create a child class Bus that will inherit all of the variables and methods of the Vehicle class

In [30]:
```
# Parent class
class Vehicle:
    def __init__(self, name, max_speed, mileage):
        self.name = name
        self.max_speed = max_speed
        self.mileage = mileage

    def display(self):
        print("Vehicle Name:", self.name)
        print("Max Speed:", self.max_speed)
        print("Mileage:", self.mileage)


# Child class inheriting Vehicle
class Bus(Vehicle):
    pass


# Creating object of Bus class
bus = Bus("Volvo Bus", 120, 10)

# Accessing inherited method
bus.display()
```

```
Vehicle Name: Volvo Bus
Max Speed: 120
Mileage: 10
```

2) Define a property that must have the same value for every class instance (object)

In [31]:
```python
class Vehicle:
    # Class variable
    wheels = 4

    def __init__(self, name):
        self.name = name

    def display(self):
        print("Vehicle Name:", self.name)
        print("Number of Wheels:", Vehicle.wheels)


# Creating objects
v1 = Vehicle("Car")
v2 = Vehicle("Truck")

v1.display()
v2.display()
```

```
Vehicle Name: Car
Number of Wheels: 4
Vehicle Name: Truck
Number of Wheels: 4
```

3) Write a Python class Employee with attributes like emp_id, emp_name, emp_salary, and emp_department and methods like calculate_emp_salary, emp_assign_department, and print_employee_details

In [32]:
```python
class Employee:
    def __init__(self, emp_id, emp_name, emp_salary, emp_department):
        self.emp_id = emp_id
        self.emp_name = emp_name
        self.emp_salary = emp_salary
        self.emp_department = emp_department

    # Method to calculate employee salary (with overtime)
    def calculate_emp_salary(self, hours_worked):
        if hours_worked > 50:
            overtime = hours_worked - 50
            self.emp_salary += overtime * (self.emp_salary / 50)
        return self.emp_salary
```

```python
    # Method to assign / change department
    def emp_assign_department(self, new_department):
        self.emp_department = new_department

    # Method to print employee details
    def print_employee_details(self):
        print("Employee ID      :", self.emp_id)
        print("Employee Name    :", self.emp_name)
        print("Employee Salary  :", self.emp_salary)
        print("Employee Dept    :", self.emp_department)
        print("-" * 30)


emp1 = Employee(101, "John", 50000, "IT")
emp2 = Employee(102, "Emma", 60000, "HR")

emp1.calculate_emp_salary(55)
emp1.emp_assign_department("Development")
emp1.print_employee_details()

emp2.print_employee_details()
```

```
Employee ID      : 101
Employee Name    : John
Employee Salary  : 55000.0
Employee Dept    : Development
------------------------------
Employee ID      : 102
Employee Name    : Emma
Employee Salary  : 60000
Employee Dept    : HR
------------------------------
```

4) Write a Python class Restaurant with attributes like menu_items, book_table, and customer_orders, and methods like add_item_to_menu, book_tables, and customer_order

In [33]:
```python
class Restaurant:
    def __init__(self):
        self.menu_items = []
        self.book_table = []
```

```python
        self.customer_orders = []

    # Method to add items to the menu
    def add_item_to_menu(self, item):
        self.menu_items.append(item)

    # Method to book a table
    def book_tables(self, table_no):
        self.book_table.append(table_no)

    # Method to take customer order
    def customer_order(self, order):
        self.customer_orders.append(order)

    # Method to display restaurant details
    def display_details(self):
        print("Menu Items:", self.menu_items)
        print("Booked Tables:", self.book_table)
        print("Customer Orders:", self.customer_orders)


# Creating object of Restaurant class
res = Restaurant()

# Adding menu items
res.add_item_to_menu("Pizza")
res.add_item_to_menu("Burger")
res.add_item_to_menu("Pasta")

# Booking tables
res.book_tables(1)
res.book_tables(3)

# Taking customer orders
res.customer_order("Pizza")
res.customer_order("Pasta")

# Display details
res.display_details()
```

```
Menu Items: ['Pizza', 'Burger', 'Pasta']
Booked Tables: [1, 3]
Customer Orders: ['Pizza', 'Pasta']
```

5) Write a Python class BankAccount with attributes like account_number, balance, date_of_opening and customer_name, and methods like deposit, withdraw, and check_balance.

In [34]:
```python
class BankAccount:
    def __init__(self, account_number, customer_name, date_of_opening, balance=0):
        self.account_number = account_number
        self.customer_name = customer_name
        self.date_of_opening = date_of_opening
        self.balance = balance

    # Method to deposit amount
    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print("Amount Deposited:", amount)
        else:
            print("Invalid deposit amount")

    # Method to withdraw amount
    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print("Amount Withdrawn:", amount)
        else:
            print("Insufficient Balance")

    # Method to check balance
    def check_balance(self):
        print("Current Balance:", self.balance)

    # Method to display account details
    def display_details(self):
        print("Account Number   :", self.account_number)
        print("Customer Name    :", self.customer_name)
        print("Date of Opening  :", self.date_of_opening)
        print("Balance          :", self.balance)
        print("-" * 30)
```

```python
# Creating object of BankAccount
acc = BankAccount(1001, "Alice", "01-01-2024", 5000)

# Performing operations
acc.deposit(2000)
acc.withdraw(1500)
acc.check_balance()
acc.display_details()
```

```
Amount Deposited: 2000
Amount Withdrawn: 1500
Current Balance: 5500
Account Number   : 1001
Customer Name    : Alice
Date of Opening  : 01-01-2024
Balance          : 5500
------------------------------
```

6) Write a Python class Inventory with attributes like item_id, item_name, stock_count, and price, and methods like add_item, update_item, and check_item_details

In [35]:
```python
class Inventory:
    def __init__(self, item_id, item_name, stock_count, price):
        self.item_id = item_id
        self.item_name = item_name
        self.stock_count = stock_count
        self.price = price

    # Method to add stock items
    def add_item(self, quantity):
        if quantity > 0:
            self.stock_count += quantity
            print("Item stock added successfully")
        else:
            print("Invalid quantity")

    # Method to update item price
    def update_item(self, new_price):
        if new_price > 0:
```

```python
            self.price = new_price
            print("Item price updated successfully")
        else:
            print("Invalid price")

    # Method to check item details
    def check_item_details(self):
        print("Item ID       :", self.item_id)
        print("Item Name     :", self.item_name)
        print("Stock Count   :", self.stock_count)
        print("Item Price    :", self.price)
        print("-" * 30)


# Creating object of Inventory class
item1 = Inventory(101, "Laptop", 10, 55000)

# Using methods
item1.add_item(5)
item1.update_item(58000)
item1.check_item_details()
```

```
Item stock added successfully
Item price updated successfully
Item ID       : 101
Item Name     : Laptop
Stock Count   : 15
Item Price    : 58000
------------------------------
```

In [ ]: